

DAY 17 — PostgreSQL Indexes

You are now entering one of the most important PostgreSQL engineering topics:

Why is my SQL query correct but slow?

Today you'll learn how PostgreSQL finds rows efficiently, when indexes help, when they don't, and how indexes interact with your existing knowledge of WHERE, JOIN, ORDER BY, and LEFT JOIN.

No beginner SQL recap.

1. What Is an Index?

Imagine this table:

documents

```
-----  
id | user_id | title | status  
-----
```

```
1 | 101 | ... | processed
```

```
2 | 205 | ... | failed
```

```
3 | 101 | ... | processing
```

```
...
```

```
100,000,000 rows
```

You run:

```
SELECT *  
FROM documents  
WHERE user_id = 101;
```

Without a useful index, PostgreSQL may need to inspect a very large portion of the table.

Conceptually:

No index

documents

↓

row 1

row 2

row 3

row 4

...

row 100,000,000

↓

check user_id

An index can provide a much faster lookup path:

Index on user_id

↓

user_id = 101

↓

matching row locations

↓

table

Think of an index as a **search structure**, not a second copy of your table that magically makes every query fast.

2. The Most Important Mental Model

A database table is optimized primarily for **storing rows**.

An index is optimized for **finding rows based on particular columns/expressions**.

For example:

```
CREATE INDEX idx_documents_user_id
ON documents(user_id);
```

Now PostgreSQL has an index specifically useful for searches involving user_id.

3. Basic Index Creation

```
CREATE INDEX idx_documents_user_id
ON documents(user_id);
```

Naming convention:

idx_<table>_<column>

For example:

```
CREATE INDEX idx_documents_status
ON documents(status);
```

4. Why Indexes Help

Suppose:

documents = 100 million rows

and:

WHERE user_id = 42

If only 500 documents belong to user 42, an index can potentially avoid examining every document.

Conceptually:

Without index:

100,000,000 candidates

With useful index:

~500 matching locations

The actual behavior depends on:

- table size
- data distribution
- statistics
- query shape
- selectivity
- physical storage
- PostgreSQL's chosen execution plan

So don't memorize:

"Index = always faster."

That is false.

5. Indexes Have a Cost

Indexes aren't free.

Suppose:

`INSERT INTO documents ...`

PostgreSQL must maintain relevant indexes too.

So:

`INSERT`

`UPDATE`

`DELETE`

↓

table changes

+

index changes

Indexes also consume storage.

Therefore:

Don't create an index on every column.

6. The Most Common PostgreSQL Index

The default index type is:

B-tree

For example:

```
CREATE INDEX idx_documents_user_id
```

```
ON documents(user_id);
```

B-tree indexes are useful for many common operations involving:

=

<

>

<=

>=

```
ORDER BY
```

range conditions

For example:

```
WHERE user_id = 100
```

or:

```
WHERE page_count > 50
```

or:

```
WHERE created_at >= ...
```

7. Primary Keys and Indexes

Remember Day 16:

```
CREATE TABLE users (
```

```
id BIGINT PRIMARY KEY
```

```
);
```

PostgreSQL automatically creates an index to enforce the primary-key uniqueness requirement.

So you generally **do not need**:

```
CREATE INDEX idx_users_id
```

```
ON users(id);
```

again.

You'd just be creating a redundant index.

8. UNIQUE Constraints and Indexes

Similarly:

```
email TEXT UNIQUE
```

requires PostgreSQL to efficiently enforce uniqueness.

PostgreSQL creates a unique index to support that constraint.

So:

```
email TEXT UNIQUE
```

already gives you an index-like structure supporting uniqueness.

Don't blindly create:

```
CREATE INDEX idx_users_email
```

```
ON users(email);
```

without understanding what already exists.

9. Indexes and WHERE

Consider:

```
SELECT *
```

```
FROM documents
```

```
WHERE user_id = 42;
```

Potentially useful:

```
CREATE INDEX idx_documents_user_id
```

```
ON documents(user_id);
```

10. Indexes and JOINS

This is especially important for Data Engineering.

You frequently write:

```
SELECT
```

```
u.id,
```

```
d.id
```

```
FROM users u
```

```
JOIN documents d
```

```
ON d.user_id = u.id;
```

A useful index on:

documents(user_id)

can help PostgreSQL efficiently locate documents for users.

This is particularly relevant because:

users

1 million rows

documents

100 million rows

is a very different situation from:

users

100 rows

documents

1,000 rows

The optimizer considers the entire situation.

11. Foreign Key ≠ Automatically Indexed Child Column

This is a very important PostgreSQL/Data Engineering point.

Suppose:

```
CREATE TABLE documents (  
  id BIGINT PRIMARY KEY,  
  user_id BIGINT REFERENCES users(id)  
);
```

The foreign key enforces referential integrity.

But you should **not assume** that PostgreSQL automatically creates an index on:
documents.user_id

just because it's a foreign key.

If you frequently do:

```
JOIN documents d  
ON d.user_id = u.id
```

you may want:

```
CREATE INDEX idx_documents_user_id  
ON documents(user_id);
```

This is one reason foreign-key design and indexing are separate concerns.

12. Indexes and ORDER BY

Suppose:

```
SELECT *  
FROM documents  
ORDER BY created_at DESC  
LIMIT 20;
```

An index can potentially help:

```
CREATE INDEX idx_documents_created_at  
ON documents(created_at);
```

PostgreSQL B-tree indexes can be scanned in either direction.

For a simple case, one ascending index can support both:

ASC

DESC

queries.

13. WHERE + ORDER BY

Now consider:

```
SELECT *  
FROM documents  
WHERE user_id = 42  
ORDER BY created_at DESC  
LIMIT 20;
```

An index on only:

`user_id`

may help find the user's documents.

An index on:

`created_at`

may help ordering.

But PostgreSQL may benefit from a **composite index**:

```
CREATE INDEX idx_documents_user_created  
ON documents(user_id, created_at DESC);
```

This is a major production concept.

14. Composite Index

Example:

```
CREATE INDEX idx_documents_user_created  
ON documents(user_id, created_at DESC);
```

The index is ordered conceptually by:

`user_id`

↓

`created_at`

This can be very useful for:

```
WHERE user_id = 42  
ORDER BY created_at DESC  
LIMIT 20;
```

15. Column Order Matters

This is critical.

Consider:

```
CREATE INDEX idx_documents_user_status  
ON documents(user_id, status);
```

The order is:

`user_id`

↓

`status`

It is not equivalent to:

```
CREATE INDEX idx_documents_status_user
ON documents(status, user_id);
```

These indexes can behave differently depending on the query.

Think about the first column as the primary navigation dimension.

16. Leftmost Prefix Intuition

Suppose:

```
CREATE INDEX idx_documents_user_status
ON documents(user_id, status);
```

This is naturally useful for queries such as:

```
WHERE user_id = 10
and:
WHERE user_id = 10
AND status = 'failed'
```

But a query only filtering:

```
WHERE status = 'failed'
```

doesn't get the same benefit from the index structure.

Don't interpret this as an absolute rule about every PostgreSQL plan; the planner has other options.

But as an index-design mental model:

The leading columns matter.

17. Index Selectivity

Consider:

documents

100 million rows

and:

status

processed → 99 million

failed → 500,000

processing → 500,000

An index on:

status

may not be very useful for:

```
WHERE status = 'processed'
```

because almost the entire table matches.

This relates to **selectivity**.

High selectivity roughly means:

The condition filters out a large portion of the table.

For example:

`user_id = 987654`

might match 20 rows out of 100 million.

That's highly selective.

18. Low Cardinality Columns

Columns with few distinct values can be poor candidates for simple indexes in many workloads.

Example:

`account_status`

active

inactive

suspended

Only three values.

But don't memorize:

"Never index low-cardinality columns."

That's also wrong.

A low-cardinality column can still be useful when:

- combined with another column
 - used in a partial index
 - the filtered value is rare
 - query patterns justify it
-

19. Partial Indexes

This is extremely useful for production RAG systems.

Suppose:

documents

contains:

processed

processing

failed

But your application frequently queries:

```
SELECT *
```

```
FROM documents
WHERE status = 'failed';
```

You could create:

```
CREATE INDEX idx_documents_failed
ON documents(user_id)
WHERE status = 'failed';
```

This is a **partial index**.

It indexes only rows satisfying:

status = 'failed'

instead of the entire table.

20. Why Partial Indexes Can Be Powerful

Suppose:

100 million documents

but only:

100,000 failed

Then a partial index focused on failed documents can be much smaller than indexing all rows.

Example:

```
CREATE INDEX idx_documents_failed_user
ON documents(user_id)
WHERE status = 'failed';
```

Useful for:

```
SELECT *
FROM documents
WHERE user_id = 42
AND status = 'failed';
```

21. Indexes and NULL

This connects to your Day 15 weakness.

Suppose:

```
CREATE INDEX idx_documents_user_id
ON documents(user_id);
```

Now consider:

WHERE user_id IS NULL

PostgreSQL indexes can contain NULL values in ordinary B-tree indexes.

So don't assume:

"NULL can't be indexed."

It can.

22. LEFT JOIN + Indexes

Consider:

```
SELECT
u.id,
d.id
FROM users u
LEFT JOIN documents d
ON d.user_id = u.id
AND d.status = 'processed';
```

Potentially useful:

```
CREATE INDEX idx_documents_user_status
ON documents(user_id, status);
```

Why?

Because the database repeatedly needs to answer:

Find documents belonging to this user where status is processed.

This is a nice example where your **join semantics** and **index design** connect.

23. Important: Index Does Not Fix Bad SQL Semantics

Suppose you write:

```
SELECT
u.id,
COUNT(d.id)
FROM users u
LEFT JOIN documents d
ON d.user_id = u.id
WHERE d.status = 'processed'
GROUP BY u.id;
```

Adding an index doesn't fix the logical problem.

The query still filters unmatched users.

Remember:

Correct semantics first

↓

Correct query

↓

Then optimize

Not:

Bad query

↓

Add index

↓

Hope it becomes correct

An index improves access paths.

It does not repair incorrect business logic.

24. EXPLAIN

Now we reach the most important tool for understanding indexes.

```
EXPLAIN
SELECT *
FROM documents
WHERE user_id = 42;
```

PostgreSQL returns a query plan.

For example, conceptually:

Index Scan using idx_documents_user_id
on documents
or:

Seq Scan on documents

25. Sequential Scan

You may see:

Seq Scan

This means PostgreSQL is scanning the table sequentially.

Conceptually:

row 1

↓

row 2

↓

row 3

↓

...

This isn't automatically bad.

For a small table:

100 rows

a sequential scan may be cheaper than using an index.

26. Index Scan

You may see:

Index Scan

Conceptually:

Index

↓

find matching row locations

↓

fetch table rows

Useful when relatively few rows match.

27. Bitmap Index Scan

You may also see:

Bitmap Index Scan

and:

Bitmap Heap Scan

This can be useful when many rows match and PostgreSQL wants to collect matching table locations efficiently.

You don't need to memorize the internals yet.

Just recognize:

Seq Scan

Index Scan

Bitmap Index Scan

as different access strategies.

28. EXPLAIN ANALYZE

EXPLAIN shows the **planned** execution strategy.

EXPLAIN ANALYZE actually executes the query and reports runtime information.

Example:

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM documents
```

```
WHERE user_id = 42;
```

You might see things like:

actual time

rows

loops

This becomes central on Day 20.

29. WARNING: EXPLAIN ANALYZE Executes the Query

This matters.

For:

```
SELECT
```

that's usually straightforward.

But:

```
EXPLAIN ANALYZE
```

```
DELETE ...
```

actually performs the deletion.

Likewise for:

```
UPDATE
```

```
INSERT
```

So in production:

Be careful with EXPLAIN ANALYZE.

30. ANALYZE and Statistics

PostgreSQL maintains statistics about tables and columns.

These help the optimizer estimate:

How many rows will match?

For example:

```
ANALYZE documents;
```

updates planner statistics.

The optimizer uses statistics to choose between things like:

Seq Scan

Index Scan

Bitmap Scan

31. Why PostgreSQL May Ignore Your Index

You create:

```
CREATE INDEX idx_documents_status  
ON documents(status);
```

Then:

```
SELECT *  
FROM documents  
WHERE status = 'processed';
```

and PostgreSQL chooses:

Seq Scan

You might think:

"PostgreSQL isn't smart."

Not necessarily.

If:

95% of rows = processed

a sequential scan may simply be cheaper.

The optimizer isn't obligated to use an index.

32. Index + LIMIT

Consider:

```
SELECT *
```

```
FROM documents
```

```
WHERE user_id = 42
```

```
ORDER BY created_at DESC
```

```
LIMIT 10;
```

A well-designed composite index can potentially make this extremely efficient:

```
CREATE INDEX idx_documents_user_created
```

```
ON documents(user_id, created_at DESC);
```

The database can potentially:

find user 42

↓

walk newest records

↓

stop after 10

This is one of the most useful production patterns.

33. Expression Index

Suppose you frequently search:

```
WHERE LOWER(email) = 'aarav@example.com'
```

An ordinary index on:

email

doesn't necessarily provide the same benefit because your query applies a function.

You can create:

```
CREATE INDEX idx_users_lower_email
```

```
ON users(LOWER(email));
```

Then:

```
WHERE LOWER(email) = 'aarav@example.com'
```

can use the expression index.

34. Indexing JSONB

PostgreSQL can index JSONB.

For example:

```
CREATE INDEX idx_documents_metadata  
ON documents
```

```
USING GIN(metadata);
```

GIN is an important PostgreSQL index type for structures such as:

JSONB

arrays

You'll encounter this frequently in GenAI systems where document metadata is stored in JSONB.

35. B-tree vs GIN

For now, remember:

B-tree

Common for:

=

<

>

<=

>=

```
ORDER BY
```

ranges

GIN

Common for:

JSONB

arrays

full-text/search-like structures

Don't try to memorize every index type yet.

36. Index Maintenance

Indexes need maintenance as data changes.

Large indexes consume:

disk space

memory/cache

write resources

PostgreSQL's MVCC architecture also means dead tuples can accumulate.

Vacuuming helps maintain database health.

We'll study PostgreSQL internals more deeply later.

DAY 17 — DIFFICULT PRACTICE

No answers yet.

Solve these yourself.

Challenge 1 — Basic Index Design

You frequently execute:

```
SELECT *  
FROM documents  
WHERE user_id = 5001;
```

The table contains:

200 million rows

What index would you create?

Write the SQL.

Then explain why.

Challenge 2 — JOIN Index

You frequently run:

```
SELECT  
u.id,  
u.name,  
d.title  
FROM users u  
JOIN documents d  
ON d.user_id = u.id  
WHERE u.id = 5001;
```

Which column would you consider indexing on documents?

Write the index.

Explain why the primary key index on:

users.id

doesn't automatically solve the lookup on:

documents.user_id

Challenge 3 — Composite Index

Your most frequent query is:

```
SELECT
  id,
  title,
  created_at
FROM documents
WHERE user_id = 5001
ORDER BY created_at DESC
LIMIT 20;
```

Design an appropriate composite index.

Then explain:

1. Why those columns?
 2. Why that order?
 3. Why might this be better than two separate indexes?
-

Challenge 4 — LEFT JOIN + Index Design

You frequently run:

```
SELECT
  u.id,
  u.name,
  COUNT(d.id) AS failed_documents
FROM users u
LEFT JOIN documents d
  ON d.user_id = u.id
  AND d.status = 'failed'
GROUP BY
  u.id,
  u.name;
```

There are:

100 million documents

but only:

0.1% failed

Design an index strategy.

Consider:

- composite index
- partial index
- join column

- status
- preserving LEFT JOIN semantics

Explain your reasoning.

Challenge 5 — Debugging Index Design

A developer creates:

```
CREATE INDEX idx_documents_status  
ON documents(status);
```

Then says:

"Now every query filtering by status will be fast."

Is that statement correct?

Give at least **three reasons** why the index might not produce a large performance improvement.

Challenge 6 — EXPLAIN Reasoning

Suppose:

```
EXPLAIN  
SELECT *  
FROM documents  
WHERE user_id = 42;
```

returns conceptually:

Seq Scan on documents

Filter: (user_id = 42)

The developer says:

"PostgreSQL ignored my index, so PostgreSQL is broken."

Give several possible explanations.

Don't assume the index is necessarily wrong.

Challenge 7 — Selectivity

Suppose:

documents = 100 million rows

Column:

status

contains:

processed = 99,000,000

failed = 500,000

processing = 500,000

Which of these queries is more likely to benefit from an index on status?

A:

```
WHERE status = 'processed'
```

or:

B:

WHERE status = 'failed'

Explain your reasoning.

Do **not** simply answer A/B.

Challenge 8 — Partial Index

You have:

```
SELECT
```

```
id,
```

```
title
```

```
FROM documents
```

```
WHERE status = 'failed'
```

```
ORDER BY created_at DESC
```

```
LIMIT 50;
```

Only 0.1% of documents are failed.

Design a partial index.

Think carefully about:

```
WHERE
```

```
ORDER BY
```

Challenge 9 — JSONB

You have:

```
CREATE TABLE documents (
```

```
id BIGINT PRIMARY KEY,
```

```
metadata JSONB
```

```
);
```

Queries frequently look like:

```
SELECT *
```

```
FROM documents
```

```
WHERE metadata @> '{"language": "en"}';
```

What PostgreSQL index type would you investigate?

Explain why.

Challenge 10 — Production Decision

You have a table:

documents

500 million rows

You are considering creating indexes on:

user_id

status

created_at

page_count

title

file_type

The developer says:

"Let's index all of them. Indexes make queries faster."

Explain why this is dangerous.

Discuss at least:

- storage
 - INSERT performance
 - UPDATE performance
 - DELETE performance
 - low selectivity
 - workload
 - composite indexes
 - actual query patterns
-

DEBUGGING ROUND

Debug Q11

Given:

```
CREATE INDEX idx_documents_user_created  
ON documents(user_id, created_at DESC);
```

Developer runs:

```
SELECT *  
FROM documents  
WHERE created_at >= NOW() - INTERVAL '7 days';
```

They complain:

"Why isn't this index necessarily the ideal index for this query?"

Explain the relationship between:

index column order

query predicates

Debug Q12

Given:

```
CREATE INDEX idx_users_email  
ON users(email);
```

Query:

```
SELECT *
```

FROM users

WHERE LOWER(email) = 'aarav@example.com';

Why might PostgreSQL not get the expected benefit from the index?

What index could be designed specifically for this query pattern?

INTERVIEW ROUND

Answer without searching.

Q13

What is an index?

Q14

Why aren't all columns indexed automatically?

Q15

What is a B-tree?

Q16

Why does a primary key already have index support?

Q17

Does creating a foreign key automatically mean the child column has an index?

Explain.

Q18

What is a composite index?

Q19

Why does column order matter in a composite index?

Q20

What is a partial index?

Q21

What is selectivity?

Q22

Why might PostgreSQL choose a sequential scan even when a suitable index exists?

Q23

Difference between:

EXPLAIN

EXPLAIN ANALYZE

Q24

Why is EXPLAIN ANALYZE potentially dangerous on UPDATE or DELETE?

INTERVIEW SCENARIO

You are interviewing for a Data Engineer role.

The interviewer says:

"Our RAG API has this query:

```
SELECT
id,
title,
created_at
FROM documents
WHERE user_id = $1
AND status = 'processed'
ORDER BY created_at DESC
LIMIT 20;
```

The table has 300 million rows and the endpoint is slow."

They ask:

"What would you investigate?"

Don't immediately say:

"Create an index."

Give me a **production debugging process**.

Your answer should consider:

1. Query correctness

2. EXPLAIN / EXPLAIN ANALYZE

3. Existing indexes

4. Data distribution

5. Selectivity

6. Composite index

7. Partial index

8. Statistics

9. Table size

10. Actual workload

Day 17 Mastery Target

You should finish today understanding this chain:

Query

↓

Predicate / JOIN / ORDER BY

↓

Possible access paths

↓

Indexes

↓

Query Planner

↓

Execution Plan

↓

Actual Performance

And especially:

"Indexes make everything faster."

"Indexes provide alternative access paths, and PostgreSQL chooses whether they're worthwhile."

Your **main Day 17 challenge** is to connect your existing SQL knowledge with performance:

WHERE

JOIN

ORDER BY

LIMIT

LEFT JOIN

↓

index design

↓

EXPLAIN

Your assignment

Solve **Challenges 1–12**, then answer **Interview Q13–24** and the **RAG API interview scenario**.

I'll review your answers without giving solutions first, and I'll specifically test whether you can reason about **composite vs partial indexes, selectivity, and LEFT JOIN performance** rather than simply memorizing index syntax.